

Single Port RAM Verification Report (SystemVerilog)

Overview

Verification of a Single Port RAM using a mailbox-based SystemVerilog testbench . Objective was to identify and fix bugs, validate functionality, execute random and directed tests, and achieve complete functional coverage.

Verification Components

- defines.sv – macro defining number of transactions (20).
- Package – includes all class files and common definitions.
- Interface – virtual interface with separate clocking blocks for Driver, Monitor and Reference Model.
- Transaction – encapsulates address, data, control and response fields.
- Generator – creates randomized and directed transactions.
- Driver – receives transactions from generator and drives DUT.
- Monitor – samples DUT outputs and forwards observations.
- Reference Model – behavioural RAM model generating expected outputs.
- Scoreboard – compares DUT output with reference model output.
- Environment – instantiates and connects all verification components using mailboxes and virtual interfaces.
- Test – builds the environment and starts verification.
- Top – instantiates DUT, interface, clock/reset generation and test.

Verification Flow

Generator → Driver → DUT → Monitor → Scoreboard

Generator → Driver → Reference Model → Scoreboard

Design Bugs Identified

During verification of the Single Port RAM, the following design issues were identified:

1. Use of Separate Read and Write Enable Signals

The design uses two independent control signals, write_enb and read_enb, to control write and read operations. In a conventional **Single Port RAM**, a single read/write control signal is typically used since only one operation (either read or write) can be performed through the single memory port in a clock cycle. The use of separate read and write enable signals is more commonly associated with **Dual Port RAM** architectures, where independent read and write operations are supported.

2. Incomplete Memory Reset

During reset, only the memory location corresponding to the current address is reset. As a result, the remaining memory locations retain their previous values,

leading to an incomplete initialization of the RAM. A proper reset implementation should initialize all memory locations to a defined state.

3. High-Impedance Output During Idle State

When neither `write_enb` nor `read_enb` is asserted, the design drives `data_out` to a high-impedance (Z) state. Consequently, the output does not retain the last successfully read value. In many RAM implementations, the read data is expected to remain stable until the next valid read operation, making the current behavior a functional limitation.

Bugs Identified and Fixed

Four-state data handling: Transaction fields declared as bit converted Z/X to 0. Changed to logic to preserve four-state values.

Missing constructor argument (Reference Model): Child class calling `super.new()` omitted the third constructor argument; corrected.

Missing constructor argument (Top/Test): Instantiation of the extended transaction/environment omitted the required third argument; corrected.

Reference model reset: Reset behaviour was missing. Added active-low reset handling so expected `data_out` becomes high impedance (8'bz), matching the DUT.

Coverage completion: Added directed stimulus to exercise missing `data_out` bit combinations and achieve complete functional coverage.

Directed Test Cases

Two directed tests were added by assigning explicit values in the generator instead of randomizing.

- Directed Test 1:
 - Perform a write to a specific address with a predefined data value.
 - Read back the same address and compare with the reference model.
- Directed Test 2:
 - Perform another write using a different address and data pattern.
 - Read back the same location and verify scoreboard match.

Special Transactions

Specific data patterns (for example 8'h00, 8'h11 and other directed values) were applied to ensure all `data_out` bits toggled between 0 and 1, improving functional coverage.

Regression

After fixing each bug, the complete test suite was rerun. Regression confirmed that previous functionality remained correct, scoreboard comparisons passed, and no new issues were introduced.

Functional Coverage

Overall functional coverage reached 100% after adding the required directed transactions.

file:///home/krishnagirish/Desktop/test/covReport/pages/_frametop.htm

documentation Forums

it started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back! Refresh Firefox...

[List of tests included in report...](#)
[List of global attributes included in report...](#)
[List of Design Units included in report...](#)

Coverage Summary by Structure:		
Design Scope	Hits %	Coverage %
top	100.00%	100.00%
intrf	100.00%	100.00%
DUV	100.00%	100.00%
ram_pkg	100.00%	100.00%
ram_transaction/copy	100.00%	100.00%
ram_transaction3/copy	100.00%	100.00%
ram_transaction4/copy	100.00%	100.00%
ram_generator/new	100.00%	100.00%
ram_generator/start	100.00%	100.00%
ram_driver	100.00%	100.00%
ram_monitor	100.00%	100.00%
ram_reference_model/new	100.00%	100.00%
ram_reference_model/start	100.00%	100.00%
ram_scoreboard/new	100.00%	100.00%
ram_scoreboard/start	100.00%	100.00%
ram_scoreboard/compare_report	100.00%	100.00%
ram_environment/new	100.00%	100.00%
ram_environment/build	100.00%	100.00%
ram_environment/start	100.00%	100.00%
ram_test/new	100.00%	100.00%
ram_test/run	100.00%	100.00%
test3/new	100.00%	100.00%
test3/run	100.00%	100.00%
test4/new	100.00%	100.00%
test4/run	100.00%	100.00%

Coverage Summary by Type:						
Total Coverage:						
					100.00%	100.00%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Covergroups	11	11	0	1	100.00%	100.00%
Statements	184	184	0	1	100.00%	100.00%
Branches	16	16	0	1	100.00%	100.00%
FEC Conditions	4	4	0	1	100.00%	100.00%
Toggles	104	104	0	1	100.00%	100.00%

Report generated by Questa (ver. 10.6c) on Tue 07 Jul 2026 05:49:26 AM IST with command line:
vcover report -html ram_cov.ucdb -htmldir covReport -details

Conclusion

The SystemVerilog verification environment successfully verified the Single Port RAM using random and directed testing. Constructor issues, four-state data handling, and reference model reset behaviour were corrected. Functional coverage reached 100% and regression testing confirmed correct DUT behaviour.